

Technologies we are going to use:

- XUL for the UI design
- JavaScript for the logic
- DOM to access HTML and the XUL interface

Setting up the development environment

First, we need to set up a file structure for our add-on, so let us begin by creating a new folder on the computer. *C:\DDG_addon* in our case is the add-on folder. Now, inside that new folder, create a folder called *content*.

In root directory of our add-on folder, create two new empty text files, called *install.rdf* and *chrome.manifest*, and in the *content* folder, create a new empty text file called *ui.xul* and an empty JavaScript file *logic.js*

You should end up with the following directory structure:

- *install.rdf*
- *chrome.manifest*
- *content*
 - *logic.js*
 - *ui.xul*

Description

install.rdf contains general information about the add-on. *chrome.manifest* registers your add-on's content with the Chrome engine.

content is the directory containing XUL and JavaScript files. *logic.js* is a JavaScript file containing the logic for the add-on. *ui.xul* is the UI design of the add-on.

Creating the install manifest

Open the *install.rdf* file and copy the following code:

```
<?xml version="1.0"?>
<RDF xmlns="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
    xmlns:em="http://www.mozilla.org/2004/em-rdf#">

  <Description about="urn:mozilla:install-manifest">
    <em:id>sample@example.net</em:id>
    <em:version>1.0</em:version>
    <em:type>2</em:type>

    <!-- Target Application this extension can install into,
        with minimum and maximum supported versions. -->
    <em:targetApplication>
      <Description>
        <em:id>{ec8030f7-c20a-464f-9b0e-13a3a9e97384}</em:id>
        <em:minVersion>1.5</em:minVersion>
        <em:maxVersion>35.0.*</em:maxVersion>
      </Description>
    </em:targetApplication>

    <!-- Front End MetaData -->
    <em:name>Duck Duck Go Search</em:name>
    <em:description>A simple yet powerful add-on</em:
    </em:description>
```

```
em:description>
  <em:creator>Aniket Kudale</em:creator>
  <em:homepageURL>http://www.aniket.co</em:homepageURL>
</Description>
</RDF>
```

Here, the code is written in XML. The file consists of general information about the add-on such as its version number, name, description, the version of the Firefox browser it supports, the creator of the add-on, etc.

- *sample@example.net* is the ID of the extension. It can be your email address.



Note: This parameter must be in the format of an email address.

- *<em:version>1.0</em:version>* specifies the version of the add-on.
- *<em:type>2</em:type>* - the value 2 declares that it is installing an add-on.
- *{ec8030f7-c20a-464f-9b0e-13a3a9e97384}* is Firefox's application ID.
- *<em:minVersion>1.5</em:minVersion>* specifies the earliest version of Firefox this add-on will work with.
- *<em:minVersion>35.0.*</em:minVersion>* specifies the minimum version of Firefox that this add-on will work with. In this case, 35.0.* indicates that the add-on works with Firefox 35.0 and its subsequent 35.0.x releases.

Adding the XUL

Firefox's user interface is written in XUL and JavaScript. XUL provides the user interface for the Firefox browser through widgets like toolbars, buttons, menus, etc. We will be using XUL Overlays for our add-on. They provide a way of attaching UI widgets to XUL documents at runtime.

Open the *ui.xul* file and copy the following code:

```
<?xml version="1.0"?>
<overlay id="Scrapper-Overlay"
    xmlns="http://www.mozilla.org/keymaster/gatekeeper/there.
    is.only.xul">
  <script type="application/x-javascript" src="chrome://DDG_
    add-on/content/logic.js" />
  <toolbar id="navigator-toolbox">
    <toolbar id="DDGToolbar" toolbarname="DuckDuckGo Toolbar"
    >
      <label value="DuckDuckGo Search: " />
      <textbox id="DDGQuery" cols="1" size="28" />
      <toolbarbutton id="DDGButton" label="Go" oncommand="D
        uckDuckGoSearch(event)" />
    </toolbar>
  </toolbar>
</overlay>
```